

딥레이서 팀 브리핑

경북대 × 전남대 Physical AI 경진대회. 트랙과 채점 방식이 확정되어 설계를 전면 개정했습니다. 처음 하시는 분도 이 문서만 따라가면 모델을 만들 수 있습니다.

$$\text{성적} = (\text{모델1 시간} + \text{모델2 시간}) \div 2$$

Best Lap이 아니라 **평균**입니다. 버릴 수 있는 주행이 없으므로 두 모델 모두 좋아야 합니다. 표준 출발 단일 랩, 탈선 시 1초 추가 후 재배치.

트랙

A to Z Speedway

16.64 m × 107 cm

본선

8/21 (금)

10:00 – 12:00

차량

모노 카메라

LiDAR 없음

방침

최대한 공격적

안 나가는 선에서

레인

5개 가설

각자 하나씩

목표

평균 9초 이내

도전 8초

01 먼저 이것만 이해하면 됩니다**02** 왜 “최대한 공격적”이 정답인가**03** 5개 레인 — 누가 무엇을 맡는가**04** 액션 스페이스란 무엇인가**05** 레인별 입력표 (그대로 타이핑)**06** 콘솔 조작 순서와 함정**07** 당일 진행과 판정

01 먼저 이것만 이해하면 됩니다

강화학습은 정답을 알려주지 않고 점수만 주고 스스로 배우게 하는 방법입니다. 우리는 차에게 “어떻게 운전하라”를 가르치지 않습니다. “무엇이 좋은 것인지”만 정의하고, 나머지는 차가 수십만 번의 시행착오로 찾아냅니다.

우리가 정의하는 것 2개

① 액션 스페이스 — 차가 고를 수 있는 (조향각, 속도) 선택지 목록

② 보상 함수 — 무엇을 잘한 것으로 볼지 정하는 파이썬 코드

이 둘만 우리가 만들고, 나머지는 콘솔 설정입니다.

차가 하는 일

초당 약 15번, 카메라 화면을 보고 목록에서 한 줄을 고릅니다. 좋은 점수를 받은 선택은 더 자주, 나쁜 점수는 덜 하게 됩니다.

목록에 없는 조합은 아예 못 합니다. 이게 중요합니다.

이번 대회 규칙이 전략을 결정합니다

규칙	우리에게 의미하는 것
2개 모델의 평균 시간	보험 모델이라는 개념이 없습니다. 예전 규칙(Best Lap)이라면 하나는 안전하게, 하나는 공격적으로 가는 게 정답이었습니다. 지금은 느린 모델이 성적을 그대로 깎는 부채입니다. 두 모델 다 좋아야 합니다.
표준 출발 단일 랩	정지 상태에서 출발해 딱 한 바퀴. 가속 손실이 0.5~0.7초 발생합니다. 여러 바퀴 돌아서 잘 나온 것을 고르는 방식이 아닙니다.
탈선 = 1초 + 재배치	차를 다시 놓으면 정지 상태에서 다시 가속해야 하므로, 실제 손실은 1초 + 약 0.8초 = 약 1.8초입니다.

딱 하나 절대 안 되는 것: 완주 실패. 탈선은 1.8초를 내면 되는 거래입니다. 하지만 한 바퀴를 못 끝내면 평균 채점에서 성적이 파괴됩니다. 공격의 상한선은 “탈선 0회”가 아니라 “절대 못 끝내지 않는 것”입니다.

02 왜 "최대한 공격적"이 정답인가

감이 아니라 계산으로 확인했습니다. 결론부터: **5회 평가에서 5번 다 완주하는 가장 높은 속도**가 수학적 최적해입니다. 별도로 몸을 사릴 필요가 없습니다.

속도 캡을 한 단계 올리면 랩타임이 줄지만 탈선 위험이 올라갑니다. 그 손익을 따졌습니다. (알려진 실차 우승 페이스에 맞춰 보정한 추정치이며, 탈선 비용 1.8초 기준)

최 고 속도	추 정 랩 타임	이 전 단 계 대 비	손익분기
2.0	10.13초	—	—
2.2	9.33초	+0.80초	탈선 0.45회 까지 남는 장사
2.4	8.69초	+0.64초	탈선 0.35회 까지
2.6	8.19초	+0.50초	탈선 0.28회 까지
2.8	7.79초	+0.40초	탈선 0.22회 까지
3.0	7.48초	+0.31초	탈선 0.17회 까지

5회 평가 전부 완주 = 탈선 확률 0.2 이하입니다. 손익분기가 매 단계 0.17~0.45회이므로, **5/5를 유지하는 한 속도를 올리는 것은 항상 이득**입니다.

그래서 속도는 사다리로 올립니다

2.4 → 5/5 통과 → 2.6 → 통과 → 2.8 → 통과 → 3.0 → 깨지는 지점이 시물 상한

실차에서는 그 상한에서 한 단계 내려서 시작합니다. 시뮬레이터에는 없는 것들이 실제 차에는 있기 때문입니다 — 서보 반응 지연, 타이어 미끄러짐, 조명 변화, 배터리 전압 강하. 시물 완주율 100%는 실차 완주를 보증하지 않습니다.

목표는 평균 9초, 도전 8초입니다. 알려진 실차 우승 페이스를 이 트랙으로 환산하면 약 10.1초입니다. 8초는 그보다 27% 빨라야 하므로 **탈선 0회를 전제**합니다. 9초는 탈선 1회를 감수해도 도달 가능한 현실적 목표입니다.

03 5개 레인 — 누가 무엇을 맡는가

5명이 서로 다른 가설을 하나씩 맡습니다. 이걸 분업이 아니라 **베틱 분산**입니다. 어느 접근이 실차에서 통할지 미리 알 수 없으니 병렬로 검증하고, 실차 테스트로 상위 2개를 골라 본선에 올립니다.

가장 큰 낭비는 5명이 같은 보상 함수의 숫자만 조금씩 바꿔 돌리는 것입니다. 5배 자원을 쓰고 1인분의 정보만 남습니다. 반드시 서로 다른 가설을 맡아주세요.

레인	검증할 가설	어떻게	속도 사다리
A 중앙선 기준선	단순하고 안정적인 게 이긴다	트랙 중앙에 가까울수록 점수. 속도 보상 없음 — 속도는 액션 스페이스로만 통제. 가장 빨리 수렴 하므로 팀의 기준점이 됩니다.	2.2
B 레이싱 라인 경로	최적 궤적이 랩타임을 만든다	미리 계산한 최적 경로 좌표 110개를 코드에 심어두고, 그 선에 가까울수록 점수. 경로를 미리 알려주는 방식.	2.4 → 2.6 → 2.8
C 속도 프로파일 일 감가속	경로보다 감가속 타이밍이 랩타임을 만든다	앞쪽 웨이포인트를 읽어 다가오는 코너가 얼마나 급한지 실시간 계산하고, 그에 맞는 목표 속도를 추종하게 함.	2.6 → 2.8 → 3.0
E 궤적 매칭 최우선	위치와 속도를 함께 주면 가장 빠르다	B와 C를 합친 것. 최적 경로의 각 지점마다 “여기서는 이 속도”까지 미리 계산해 함께 매칭. 가장 정보량이 많은 보상입니다.	2.6 → 2.8
D 워블 억제 실차 대비	시물 1등이 실차 1등이 아니다	조향을 $\pm 20^\circ$ 로 제한하고 핸들을 급하게 꺾는 것 자체에 벌점. 실차에서 지그재그로 떨어리는 현상을 막습니다.	2.2

레인 D는 랩타임이 느려도 절대 지우지 마세요. D의 가치는 시물 기록이 아니라 **실차에서 흔들리지 않는 것**입니다. 오후 실차 테스트에서 B·C·E가 워블로 무너지면 D가 유일한 대안이 됩니다.

보상 함수 코드는 어디에

깃허브 private 레포 TaeHuiKKIM/deepracer-physical-ai 의 reward_functions/ 폴더에 5개 파일이 있습니다. **파일을 열어 전체 복사 → 콘솔에 붙여넣기**만 하면 됩니다. 각 파일 맨 위에 그 레인의 가설과 튜닝 방법이 주석으로 적혀 있습니다.

04 액션 스페이스란 무엇인가

가장 많이 오해하는 부분입니다. 이건 **속도 제한표가 아니라 “차가 고를 수 있는 선택지 메뉴”**입니다.

표의 한 줄 = 하나의 행동입니다. 예를 들어 **-5° / 2.2** 줄은 “핸들을 왼쪽 5도 꺾으면서 **동시에** 2.2 m/s로 달린다”는 **하나의 묶음**입니다. 각도와 속도를 따로 정하는 게 아니라 세트로 붙어 있고, 차는 매 순간 이 목록에서 한 줄을 고릅니다.

표에 없는 조합은 차가 못 합니다. 예를 들어 우리 표에는 “25도로 꺾으면서 2.6 m/s” 같은 줄이 없습니다. 그러니 차는 그렇게 하고 싶어도 **할 수 없습니다.** 즉 각도별 속도 제한이 규칙으로 적혀 있는 게 아니라, **그 조합을 메뉴에서 빼버림으로써** 만들어집니다. 보상으로 “급하게 꺾으면 벌점”이라고 가르치는 건 차가 시행착오로 배워야 하지만, 메뉴에서 빼면 처음부터 불가능합니다.

왜 레인마다 조향 범위가 다른가

계산해보니 **최적 경로가 요구하는 최대 조향각은 11.8°**이고, 랩의 55%가 5° 이내였습니다. 그래서 **경로를 따라가는 레인 B·E는 최대 20°로 좁히고 0° 근처에 해상도를 몰아줬습니다.** 넓은 각도 선택지는 경로 추종에 쓰이지 않으면서, 차가 “느리면서 많이 꺾는” 나쁜 습관을 배울 함정이 되기 때문입니다.

반면 **중앙선을 달리는 레인 A·C는 더 큰 조향이 필요합니다.** 중앙선의 코너가 레이스 라인보다 더 조이기 때문입니다(최소 반경 0.61m 대 0.79m).

개수는 8~12개

각 줄은 차가 “이 화면에서 이 행동이 좋은가”를 따로 학습해야 하는 항목입니다. 30개면 탐색 공간이 너무 커져 짧은 학습으로 수렴하지 못하고, 4~5개면 너무 거칠어 정밀한 코너링이 안 됩니다.

오른쪽 Radial polar graph로 계산

표를 그림으로 그린 것입니다. **화살표 방향 = 조향각, 길이 = 속도.** 위쪽(직진) 화살표가 가장 길고 바깥쪽(큰 조향) 화살표가 짧으면 제대로 입력된 것입니다. 바깥 화살표가 길면 “급조향 + 고속” 조합이 들어간 것이니 고쳐야 합니다.

05 레인별 입력표 — 그대로 타이핑

모두 공통: 트랙 A to Z Speedway · Time trial · 평가 5회 · 센서 Camera만(LiDAR 해제) · PPO · Batch 64 · **Epochs 3** · LR 0.0003 · Entropy 0.01 · **Discount 0.995** · Loss Huber · Episodes 20 · 학습 45~60분

기본값 그대로 두면 안 되는 것 두 개. **Number of epochs는 10 → 3** (같은 경험을 열 번 복습하면 그 상황에만 맞춰진 정책이 되어 처음 보는 코너에서 튕겨나갑니다). **Discount factor는 0.99 → 0.995** (약 200스텝 ≈ 13초 앞을 보게 되어, “지금 감속해서 손해 보고 코너를 통과해 이득 본다”는 판단을 배울 수 있습니다).

레인 A · 중앙선 캡 2.2

#	조향	속도	#	조향	속도
0	-25°	1.5	5	+5°	2.2
1	-15°	1.8	6	+10°	2.0
2	-10°	2.0	7	+15°	1.8
3	-5°	2.2	8	+25°	1.5
4	0°	2.2	9	0°	1.8

레인 B · 레이싱 라인 / 레인 E · 궤적 매칭 캡은 조향

아래는 캡 2.6 기준입니다. 사다리를 올릴 때 **CAP 표시된 칸과 Maximum speed만** 바꾸세요 (2.4 / 2.6 / 2.8).

#	조향	속도	#	조향	속도
0	0°	CAP 2.6	5	+13°	2.2
1	+4°	2.6	6	-13°	2.2
2	-4°	2.6	7	+20°	1.8
3	+8°	2.4	8	-20°	1.8
4	-8°	2.4	9	0°	2.0

레인 C · 속도 프로파일 12개

캡을 바꿀 때 **보상 함수 안의 target 첫 값도 함께** 바뀌어야 합니다(캡과 일치해야 함).

#	조향	속도	#	조향	속도
0	-28°	1.4	6	+4°	2.4
1	-20°	1.6	7	+8°	2.2
2	-14°	1.9	8	+14°	1.9
3	-8°	2.2	9	+20°	1.6
4	-4°	2.4	10	+28°	1.4
5	0°	CAP 2.6	11	0°	2.0

레인 D · 워블 억제 8개 · ±20°

#	조향	속도	#	조향	속도
0	-20°	1.4	4	+6°	2.0
1	-12°	1.7	5	+12°	1.7
2	-6°	2.0	6	+20°	1.4
3	0°	2.2	7	0°	1.6

06 콘솔 조작 순서와 합정

- 1 **Step 1 — 이름과 트랙.** 모델 이름은 {레인}-v{번호} 형식으로 (예: E-v1). 트랙 검색창에 A to Z 를 입력하고 길이 16.64m · 폭 107cm 표시를 확인하세요. 평가 횟수는 5.

합정 ① — 2018 로 검색하면 **re:Invent 2018**(17.6m·76cm)이 나옵니다. **그건 우리 트랙이 아닙니다.** 두 카드는 모양이 똑같아서 썸네일로는 구분이 안 되니 반드시 숫자를 보세요. A to Z 카드 설명에 “re:Invent 2018”이라는 문구가 들어 있어 더 헛갈립니다.

- 2 **Step 2 — 센서와 하이퍼파라미터.** 센서는 **Camera**만(LiDAR 체크 해제 — 기본값이 이미 맞습니다). 알고리즘 PPO. Hyperparameters 를 펼쳐 **Epochs 3, Discount factor 0.995**로 변경.
- 3 **Step 3 — 액션 스페이스.** **Discrete action space** 클릭(기본값은 Continuous이니 반드시 바꾸세요) → **Maximum speed**를 먼저 레인 캡으로 설정 → **Advanced configuration** 토글 ON → 05절 표대로 각 행 입력. 행 추가는 + Add an action , 삭제는 오른쪽 × .

합정 ② 가장 중요합니다 — 값을 입력한 뒤 **반드시 Tab 키를 누르세요.** Tab 없이 다음 칸으로 넘어가면 화면상 입력된 것처럼 보여도 **이전 값으로 전부 되돌아갑니다.** 한 칸 입력 → Tab → 다음 칸.

합정 ③ — Advanced configuration 을 켜지 않으면 콘솔이 자동으로 격자를 만드는데, 거기엔 **큰 조향각에 최고 속도를 붙인 줄**이 그대로 들어갑니다(예: 30°/2.6). 실차 스피드의 직행 코스입니다. 반드시 켜고 직접 입력하세요.

합정 ④ — Maximum speed 기본값이 1입니다. 먼저 바꾸지 않으면 행 속도를 그 위로 입력할 수 없습니다.

- 4 **Step 4 — 차량 외형.** 성능에 영향 없습니다. 다만 **각자 다른 색**을 고르면 학습 영상에서 누구 모델인지 바로 구분됩니다.
- 5 **Step 5 — 보상 함수.** 기본 예제 코드를 **전체 선택 후 지우고** 레포의 파일 내용을 붙여넣기 → **Validate** 클릭 → Errors: 0 확인 → **Maximum time 45~60분** 설정 → Train your model .

합정 ⑤ — 보상 함수는 PEP8 검사를 통과해야 합니다. def reward_function 앞에 **빈 줄이 정확히 2개** 필요합니다 (expected 2 blank lines, found 1 오류). 레포 파일은 이미 맞춰져 있으니 **파일 전체를 그대로 복사**하면 문제없습니다.

합정 ⑥ — Maximum time을 1440분까지 넣을 수 있지만 **절대 그러지 마세요.** 3시간을 넘기면 시뮬레이터의 배경 픽셀까지 암기해서 실차에서 무너집니다.

07 당일 진행과 판정

기록 규칙 — 이게 없으면 수십 번의 학습이 사라집니다

- **모델 이름:** {레인}-v{번호} — 이름만 보고 어떤 가설의 몇 번째 시도인지 알 수 있게
- **공용 시트 1행 = 학습 1회:** 레인 / 버전 / 이전 버전에서 바꾼 것 하나 / 평가 완주율 / 평균 랩타임 / 실차 결과
- **한 번에 하나만 바꿉니다.** 보상 함수와 속도를 동시에 바꾸면 무엇이 효과였는지 영원히 알 수 없습니다. 1순위로 바꿀 것은 항상 속도입니다.

Gate 1 · 오전 — 판정표

5회 중 완주	판정	행동
5/5	합격	Clone해서 속도만 한 단계 올린다. 사다리를 계속 올라갑니다.
3~4/5	경계	Clone해서 속도만 -0.2. 보상 함수는 그대로. 가설은 아직 살아 있습니다.
0~2/5	불합격	Clone해서 속도 -0.4. 두 번 연속 실패하면 그 레인은 접고 A 레시피로.

Gate 2 · 오후 실차 — 진짜 선발전

순위는 실물 트랙에서 갑니다. 오전이 늦어져도 오후 실차 시간은 절대 놓치지 마세요.

측정 항목	방법
① 탈선 실비용	일부러 한 번 이탈시켜 정지 → 재배치 → 원래 속도 회복까지 스톱워치로 3회 측정. 본선 공격 수위를 정하는 유일한 근거입니다.
② 워블 임계 속도	스로틀을 낮게 시작해 0.2 m/s 단위로 올리며 지그재그가 시작되는 지점 기록. 본선은 그보다 한 단계 아래로.
③ 배율별 랩타임	스로틀 80 / 90 / 100%에서 랩타임과 탈선 횟수. 본선 배율을 여기서 그대로 고릅니다.

가져갈 것: 모델 파일(Actions → Download physical car model 로 미리 내려두기), 줄자, 스톱워치, 완충 배터리, 기록용 노트.

선발 기준은 랩타임이 아니라 최종 시간입니다. 랩타임 + 탈선횟수 × 실비용 으로 줄을 세우고, 기대 시간이 가장 낮은 2개를 본선에 올립니다. 평균 채점이므로 편차가 큰 모델은 같은 평균이라도 탈락시킵니다. 트랙 시간이 짧으면 빠른 쪽부터: E → C → B → D → A.

Gate 3 · 밤 — 본선 모델 2개 확정

절대 규칙: 실차 검증을 받지 않은 모델은 본선에 올리지 않습니다. Day 3 아침에는 검증할 시간이 없고, 미검증 모델을 올리는 것은 순위가 아니라 기록 자체를 거는 도박입니다.

아직 운영진에게 확인할 것

- 완주 실패 시 어떤 시간이 기록되는가 — 평균 채점에서 최대 리스크. 상한 시간이 들어가면 사다리 상단을 얼마나 밀지가 달라집니다
- 두 모델이 서로 달라야 하는가 — 같은 모델을 2개로 등록 가능한가
- 탈선 재배치 중 시계가 멈추는가

- **역대 기록** — 목표를 8초로 잡을지 9초로 잡을지 조준용
- **트랙 방향** — A to Z가 반시계 전용인지

딥레이서 팀 브리핑 v5 · 2026-08-19 · 상세 계산 근거는 `plans/atoz_design.md`, 보상 함수는 `reward_functions/` · 깃허브 `TaeHuiKKIM/deepracer-physical-ai`